



REPTE PARADIGMES DE LA PROGRAMACIÓ I BASES DE DADES

Seminari de Bases de dades

Al. Eric Torrentera Ruiz

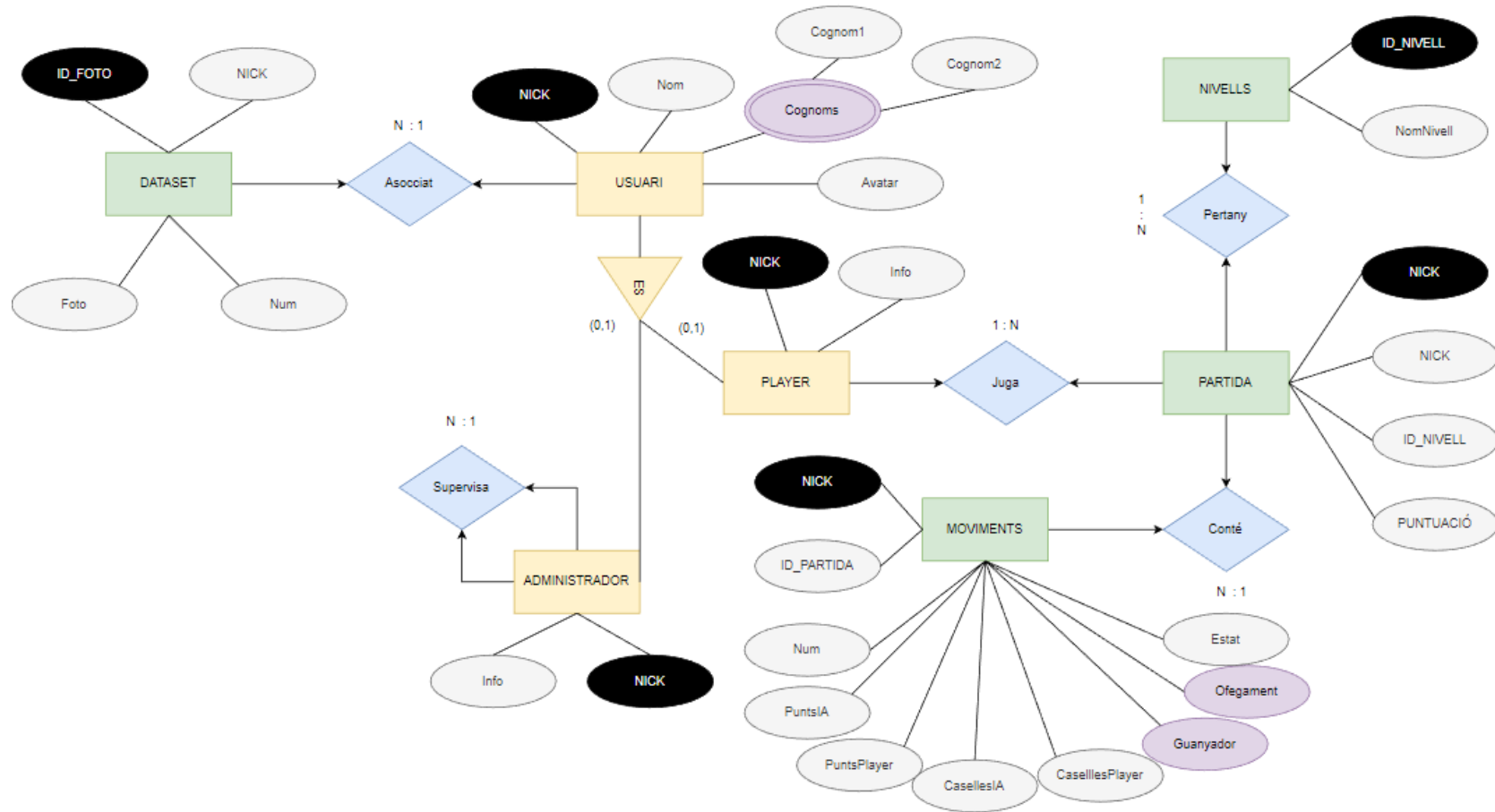
Base de dades del Joc Splatoon

La universitat d'Andorra ens demana dissenyar un joc. Pel que fa al seminari de bases de dades es vol implementar el reconeixement facial per tal de poder entrar en el joc sent reconegut directament per l'aplicació. Des del joc es podrà fer una gestió de dades referents als usuaris, puntuacions i partides efectuades.

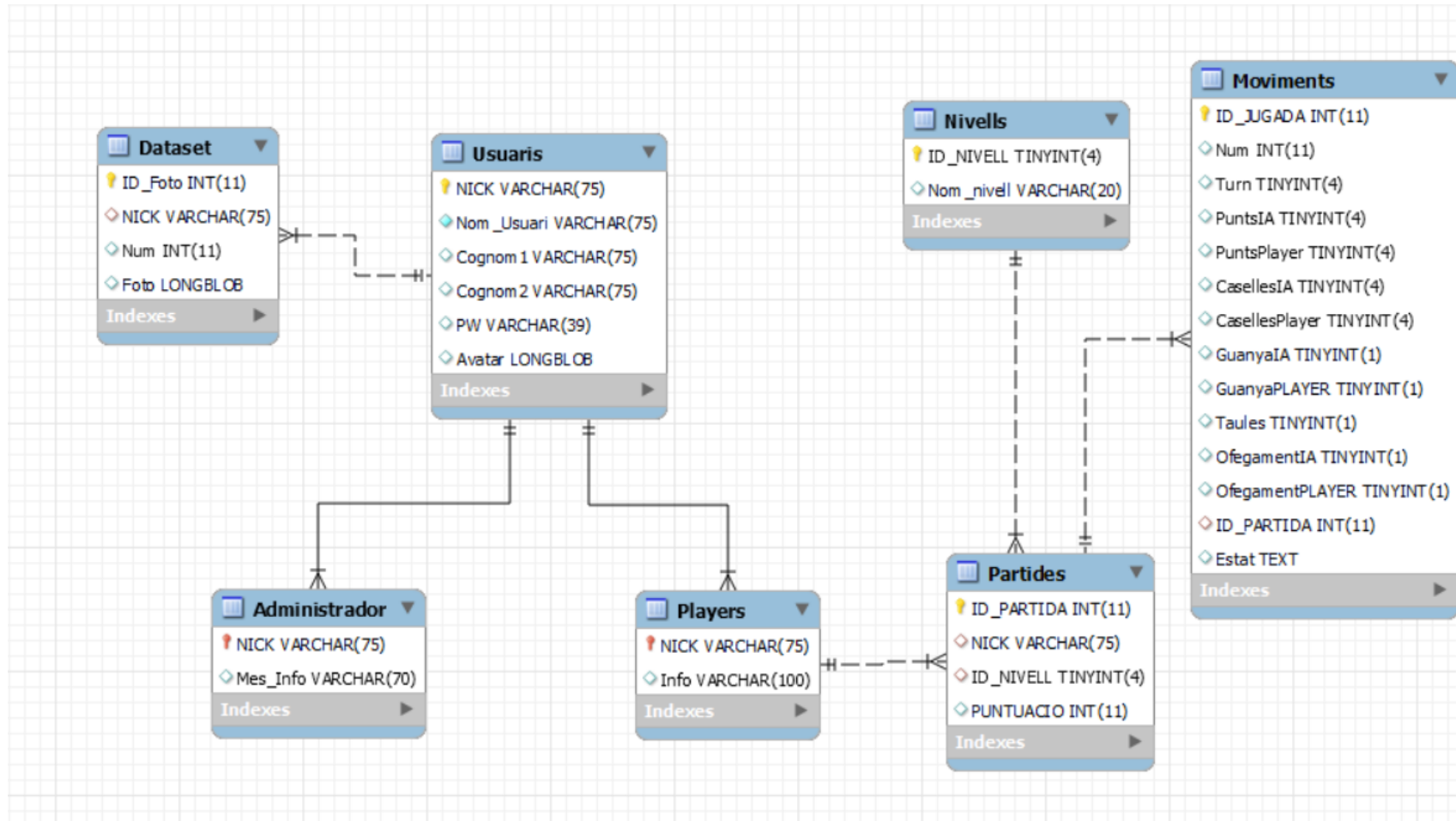
Funcionalitats demanades :

- Es vol que en el menú principal del joc, es pugui reconèixer la persona que vol jugar una partida de forma automàtica (Captura de vídeo, indicant el seu nom o persona encara desconeguda).
- En cas de ser una persona desconeguda, l'aplicació donarà l'opció de poder donar-se d'alta demanant que entri el seu nom complert i un identificador (únic, pot-ser l'email). Es farà un procés de captura facial (unes 50 imatges) que seran emmagatzemades en el servidor (MariaDB).
- A cada nova alta, el sistema ha de regenerar els arxius necessaris per poder reconèixer el nou usuari.
- En el menú ha d'existir una opció per veure els resultats obtinguts per l'usuari i també el Rànkig dels 5 millors (amb les fotos de les cares i la puntuació màxima)
- El sistema de puntuació i dades a emmagatzemar vindrà definit pels criteris enunciats en la part IA.
- Si el sistema reconeix un administrador, li proposarà poder accedir a un apartat 'gestió' que permetrà modificar o eliminar dades en les taules.

MODEL ER



Model Relacional



Passos efectuats

En primer lloc, cal transformar les entitats del diagrama amb el qual treballem en relacions. Les entitats sempre donen lloc a relacions.

Després s'ha de continuar transformant en relacions les entitats que presenten algun tipus d'especificitat (és a dir, les febles, les associatives, o les derivades d'un procés de generalització o especialització).

A continuació, s'han d'afegir a les anteriors relacions els atributs necessaris per formar les claus foranes derivades de les interrelacions binàries amb connectivitat 1-1 i 1-N presents en el diagrama ER. Les interrelacions binàries de connectivitat 1-1 o 1-N originen claus foranes en relacions ja existents.

Finalment, ja pot començar la transformació de les interrelacions binàries amb connectivitat M-N i de les interrelacions n-àries. Les interrelacions binàries de connectivitat M-N i totes les n-àries d'ordre superior a 2 sempre es transformen en noves relacions.

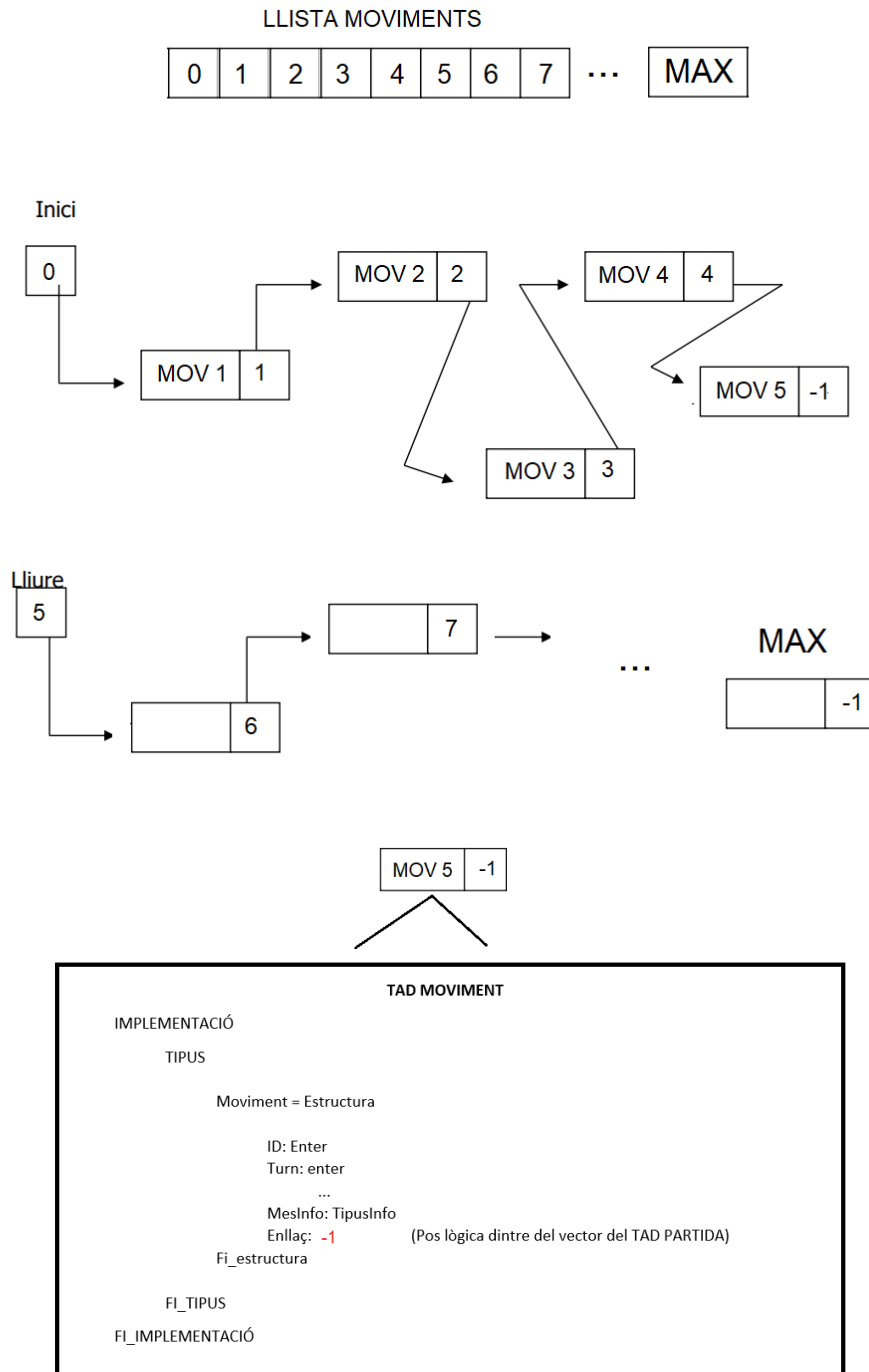
NORMALITZACIÓ DEL MODEL RELACIONAL

Hem de transformar Usuari a 1FN (Cognoms).

Hem comprovat dependències transitives i que totes les taules estan en 3FN i FNBC sense clau composta.

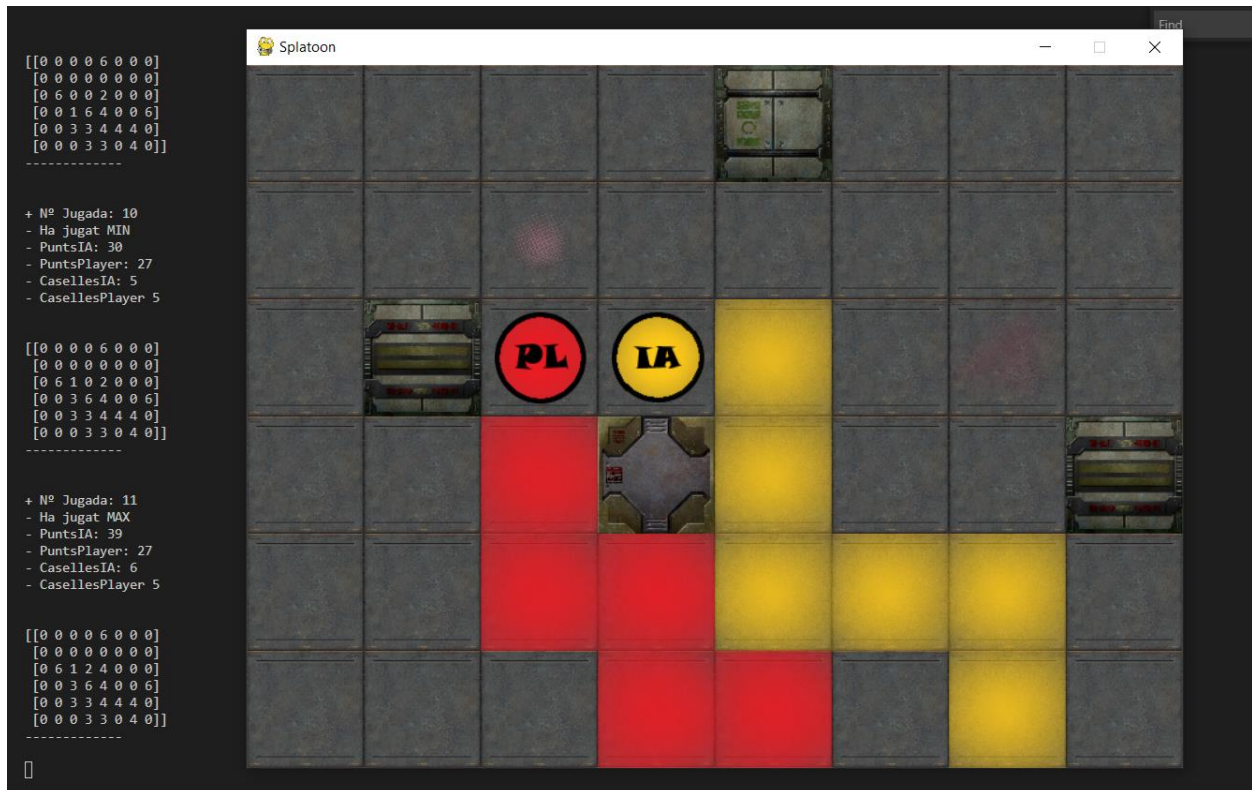
Finalment hem creat el script i hem extret el diagrama del Model Relacional amb enginyeria inversa.

ESQUEMA DEL LOE D'UNA PARTIDA AMB 5 MOVIMENTS



DESENVOLUPAMENT D'UNA PARTIDA

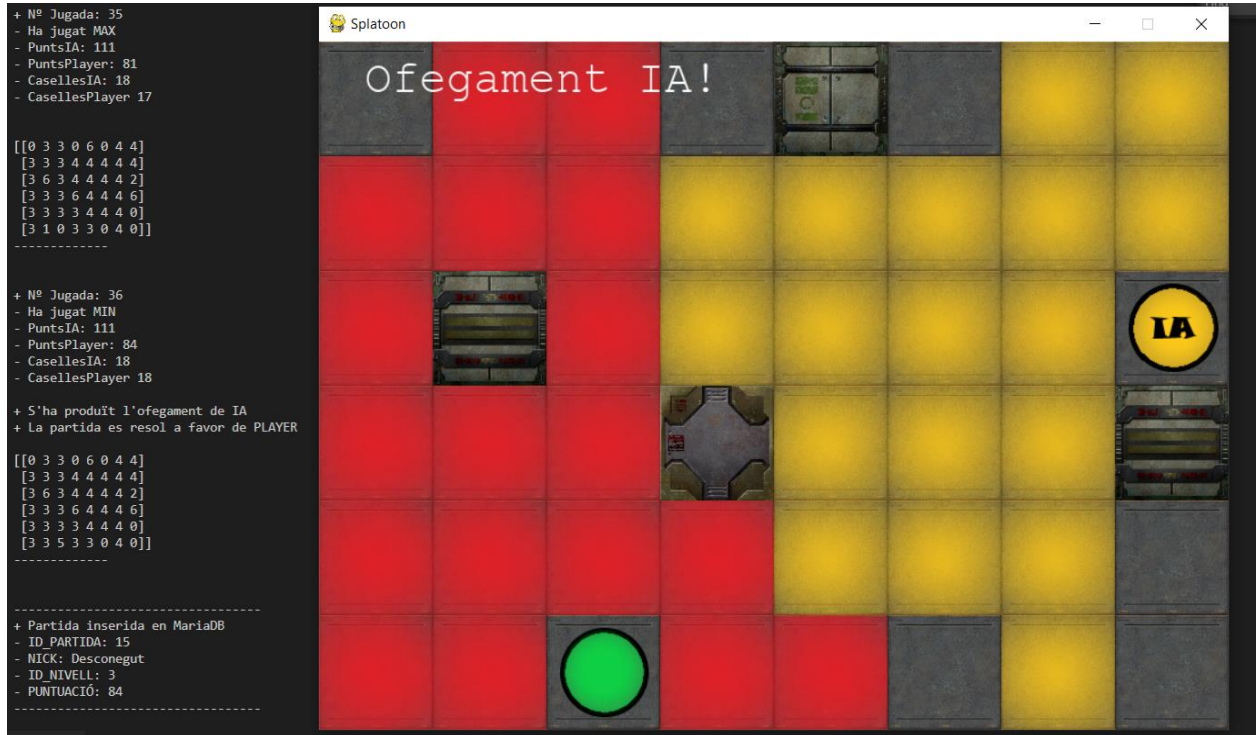
Una vegada estem jugant una partida, primerament crearem una partida i seguidament generarem els Moviments que seran emmagatzemats en una Partida (Imatge part esquerre)



Durant l'execució de Minimax crearem els moviments i emmagatzemarem la seva Info en el LOE de PARTIDA, en funció dels condicionals.

```
# PUNTERS #
PartidaIA = TADS.DonamMovimentlliure(Part.LlistaMoviments) # Agafem nou node per emmagatzemar la partida de IA
Part.LlistaMoviments.MEMO[PartidaIA].ID = num_jugada
Part.LlistaMoviments.MEMO[PartidaIA].Turn = turn
Part.LlistaMoviments.MEMO[PartidaIA].CasellesIA = info[1]
Part.LlistaMoviments.MEMO[PartidaIA].CasellesPlayer = info[2]
Part.LlistaMoviments.MEMO[PartidaIA].PuntsIA = info[3] * nivell
Part.LlistaMoviments.MEMO[PartidaIA].PuntsPlayer = info[4] * nivell
# PUNTERS #
```

Una vegada finalitzada la partida, la partida en si mateixa l'enviarem al servidor Andromeda de la universitat d'Andorra a la DB tericctest.



```

mycursor.execute("insert into Partides(NICK, ID_NIVELL, PUNTUACIO) values(%s, %s, %s)",
(NickUsuari, int(Part.ID_Nivell), int(Part.Puntuacio),))
mydb.commit()

mycursor.execute('select MAX(ID_PARTIDA) from Partides')
myresult = mycursor.fetchone()
ID_Partida = myresult[0]
  
```

```

-----
+ Partida inserida en MariaDB
- ID_PARTIDA: 15
- NICK: Desconegut
- ID_NIVELL: 3
- PUNTUACIÓ: 84
-----
  
```

```
SELECT * FROM Partides WHERE ID_PARTIDA = 15
```

ID_PARTIDA	NICK	ID_NIVELL	PUNTUACIO
15	Desconegut	3	84
NULL	NULL	NULL	NULL

GUARDEM ELS MOVIMENTS EN LA PARTIDA GUARDADA ID_PARTIDA

```
# Itera tots els moviments de la partida fins arribar al resultat final.
while x != TADS.Fi(Part.LlistaMoviments):

    mapa = str((Part.LlistaMoviments.MEMO[x].Estat).tolist())

    mycursor.execute( # Insereix totes les dades del Moviment
        """insert into Moviments(Num, Turn, PuntsIA, PuntsPlayer, CasellesIA, CasellesPlayer,
        GuanyaIA, GuanyaPLAYER, Taules, OfegamentIA, OfegamentPLAYER, ID_PARTIDA, Estat)
        values(%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)"""%(
        Part.LlistaMoviments.MEMO[x].ID,
        Part.LlistaMoviments.MEMO[x].Turn,

        int(Part.LlistaMoviments.MEMO[x].PuntsIA),
        int(Part.LlistaMoviments.MEMO[x].PuntsPlayer),

        Part.LlistaMoviments.MEMO[x].CasellesIA,
        Part.LlistaMoviments.MEMO[x].CasellesPlayer,

        Part.LlistaMoviments.MEMO[x].GuanyaIA,
        Part.LlistaMoviments.MEMO[x].GuanyaPLAYER,

        Part.LlistaMoviments.MEMO[x].Taules,
        Part.LlistaMoviments.MEMO[x].OfegamentIA,
        Part.LlistaMoviments.MEMO[x].OfegamentPLAYER,

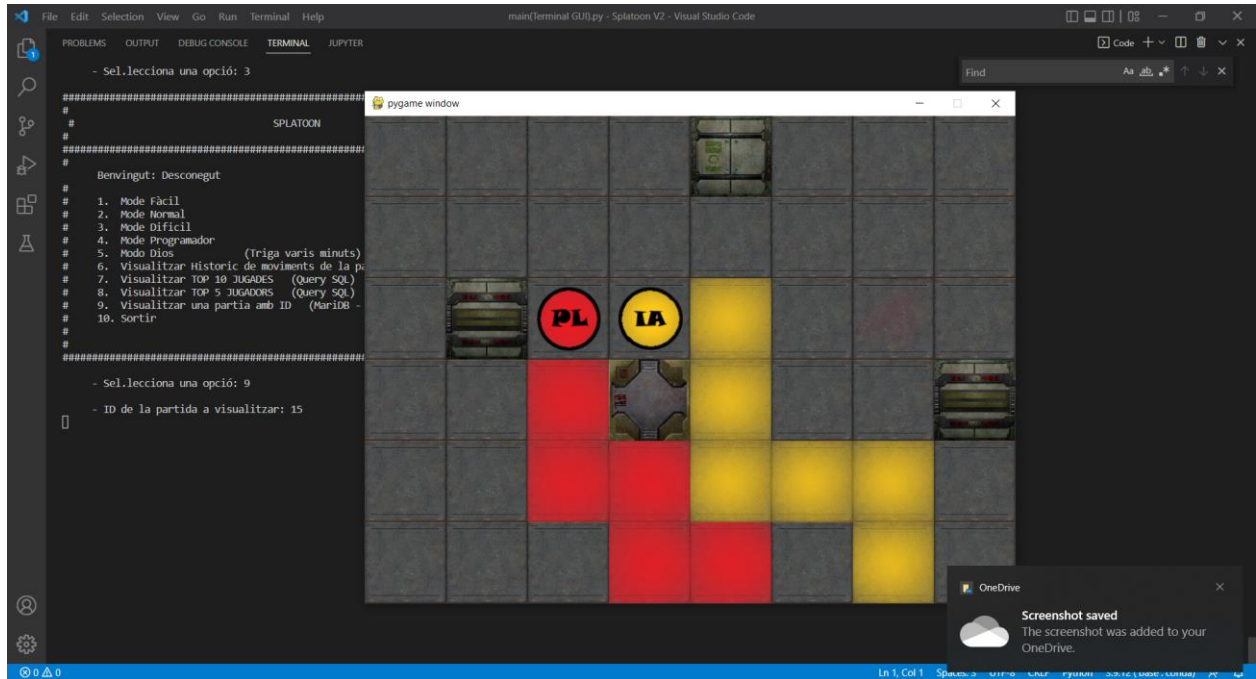
        ID_Partida,
        mapa))
    mydb.commit()

    x = TADS.Seguent(Part.LlistaMoviments, x)
```

Línia 129 MenuJocs.py

ID_JUGADA	Num	Turn	PuntsIA	PuntsPlayer	CasellesIA	CasellesPlayer	GuanyaIA	GuanyaPLAYER	Taules	OfegamentIA	OfegamentPLAYER	ID_PARTIDA	Estat
234	0	-1	0	0	0	0	0	0	0	0	0	15	[[0, 0, 0, 0, 1, 0, 2, 0], [0, 0, 0, 0, 0, 0, 0, 0],
235	1	1	3	0	1	0	0	0	0	0	0	15	[[0, 0, 0, 0, 1, 0, 4, 0], [0, 0, 0, 0, 0, 0, 2, 0],
236	2	0	3	3	1	1	0	0	0	0	0	15	[[0, 0, 0, 1, 3, 0, 4, 0], [0, 0, 0, 0, 0, 0, 2, 0],
237	3	1	9	3	2	1	0	0	0	0	0	15	[[0, 0, 0, 1, 3, 0, 4, 0], [0, 0, 0, 0, 0, 2, 4, 0],
238	4	0	9	6	2	2	0	0	0	0	0	15	[[0, 0, 0, 3, 3, 0, 4, 0], [0, 0, 0, 1, 0, 2, 4, 0],
239	5	1	15	6	3	2	0	0	0	0	0	15	[[0, 0, 0, 3, 3, 0, 4, 0], [0, 0, 0, 1, 2, 4, 4, 0],
240	6	0	15	12	3	3	0	0	0	0	0	15	[[0, 0, 0, 3, 3, 0, 4, 0], [0, 0, 1, 3, 2, 4, 4, 0],

Si pressionem la opció 9, amb l'ID d'una partida podem veure el seu desenvolupament sense jugar-la.



```
elif Opcio == 9:

    ID_Partida = input("    - ID de la partida a visualitzar: ")
    ID_Partida = int(ID_Partida) # Casting

    mycursor.execute("SELECT Estat FROM Moviments WHERE ID_PARTIDA = %s", (ID_Partida, ))
    myresult = mycursor.fetchall()

    flag_init = 1

    for x in myresult:

        Map = eval(x[0])
        Map = np.stack(Map, axis = 0) # Passem el text a array.
        # Map= np.flip(Map, axis = 0) # Com està copiat al revés l'hem de girar

        # Calculem el tamany que tenia la partida que volem carregar
        if flag_init == 1:

            col_count = len(Map[0])
            row_count = len(Map)
            width = len(Map[0]) * Constants.SQUARESIZE
            height = len(Map) * Constants.SQUARESIZE
            size = (width, height)
            screen2 = pygame.display.set_mode(size)
            flag_init = 0

        LògicaJoc.dibuixa_tauler(Map, screen2, height, col_count, row_count, True)
        pygame.time.wait(1000)
```

Línia 160 MenuJocs.py

Si pressionem la opció 7, posem veure un llistat de les 10 millors partides.

```
#####
#
#                               SCOREBOARD                               #
#
#####

ID: 6      Puntuació: 112      Nivell: 4      + jugador: Admin
ID: 1      Puntuació: 80       Nivell: 4      + jugador: Admin
ID: 9      Puntuació: 72       Nivell: 4      + jugador: Desconegut
ID: 7      Puntuació: 60       Nivell: 4      + jugador: Desconegut
ID: 11     Puntuació: 16       Nivell: 4      + jugador: Desconegut
ID: 15     Puntuació: 84       Nivell: 3      + jugador: Desconegut
ID: 3      Puntuació: 27       Nivell: 3      + jugador: Whiterose
ID: 4      Puntuació: 36       Nivell: 2      + jugador: Desconegut
ID: 5      Puntuació: 22       Nivell: 1      + jugador: Desconegut

- Pesione ENTER para continuar
```

```
if opcio == 7:

    print("#####")
    print("#")
    print("#                               SCOREBOARD                               #")
    print("#")
    print("#####")
    print()

    mycursor.execute("""SELECT NICK, PUNTUACIO, ID_NIVELL, ID_PARTIDA FROM Partides WHERE PUNTUACIO > 0 ORDER BY 3 DESC, 2 DESC, 1, 4 DESC LIMIT 10 """)
    myresult = mycursor.fetchall()

    for partida in myresult:
        print("ID: " + str(partida[3]) + "      Puntuació: " + str(partida[1]) + "      Nivell: " + str(partida[2]) + "      + jugador: " + partida[0])

    print()
    input(" - Pesione ENTER para continuar ")
    print()
```

Si pressionem la opció 8 podem veure un TOP 10 dels millors jugadors

```
#####
#
#                                TOP 5 JUGADORS                                #
#                                #                                #
#####

Puntuació: 112      Nivell: 4      + jugador: Admin
Puntuació: 72       Nivell: 4      + jugador: Desconegut
Puntuació: 27       Nivell: 3      + jugador: Whiterose

- Pesione ENTER para continuar
```

```
elif opcio == 8:

    print("#####")
    print("#                                #")
    print("#                                #")
    print("#                                #")
    print("#                                #")
    print("#####")
    print()

    mycursor.execute("""SELECT NICK, ID_NIVELL, MAX(PUNTUACIO) FROM Partides WHERE ID_PARTIDA IN
    (SELECT ID_PARTIDA FROM Partides WHERE (NICK, ID_NIVELL) IN
    (SELECT NICK, MAX(ID_NIVELL) FROM Partides WHERE PUNTUACIO > 0 GROUP BY NICK)
    AND PUNTUACIO > 0)
    GROUP BY NICK ORDER BY 2 DESC, 3 DESC""")

    myresult = mycursor.fetchall()

    for partida in myresult:
        print("      Puntuació: " + str(partida[2]) + "      Nivell: " + str(partida[1]) + "      + jugador: " + partida[0], )

    print()
    input("      - Pesione ENTER para continuar ")
    print()
```

MENÚ ADMINISTRADOR

El menú administrador només li proposarà als usuaris amb estatus Admin accedir a un apartat de gestió que permetrà modificar o eliminar dades en les taules:

Menú usuari normal:

- Starting facial recognition. Please wait.

+ Cara detectada:

- Nick: Whiterose

+ Cara detectada:

- Nick: whiterose

+ Cara detectada:

- Nick: Whiterose

+ Cara detectada:

- Nick: Whiterose

+ Cara detectada:

- Nick: Whiterose

+ Verificació completada amb èxit!



```
#####
#                                     #
#                                     SPLATOON                                     #
#                                     #
#####
#                                     #
#                                     Benvingut: Whiterose                       #
#                                     #
#                                     #
# 1. Mode Fàcil                                     #
# 2. Mode Normal                                     #
# 3. Mode Dificil                                     #
# 4. Mode Programador                                 #
# 5. Modo Dios                                     (Triga varis minuts)             #
# 6. Visualitzar Historic de moviments de la partida anterior (Punters)         #
# 7. Visualitzar TOP 10 JUGADES (Query SQL)                                     #
# 8. Visualitzar TOP 5 JUGADORS (Query SQL)                                     #
# 9. Visualitzar una partia amb ID (MariDB - Andromeda)                         #
# 10. Sortir                                         #
#                                     #
#                                     #
#####
```

Menú Admin:



- Starting facial recognition. Please wait.

+ Cara detectada:
- Nick: Admin
+ Cara detectada:
- Nick: Admin
+ Cara detectada:
- Nick: Admin
+ Cara detectada:
- Nick: Admin
+ Cara detectada:
- Nick: Admin
+ Cara detectada:
- Nick: Admin

+ Verificació completada amb èxit!

```
#####
#                                     #
#                                     SPLATOON                                #
#                                     #
#####
#                                     #
#      Benvingut: Admin                                     #
#                                     #
#      1. Mode Fàcil                                         #
#      2. Mode Normal                                       #
#      3. Mode Difícil                                       #
#      4. Mode Programador                                   #
#      5. Modo Dios           (Triga varis minuts)          #
#      6. Visualitzar Historic de moviments de la partida anterior (Punters) #
#      7. Visualitzar TOP 10 JUGADES   (Query SQL)          #
#      8. Visualitzar TOP 5 JUGADORS   (Query SQL)          #
#      9. Visualitzar una partia amb ID   (MariDB - Andromeda) #
#      10. Sortir                                           #
#                                     #
#####
#                                     #
#      11. Menú administrador                               #
#                                     #
#####
```

```
#####
#                                                                 #
#              ADMIN MENU  -  CRUD                               #
#                                                                 #
#####
#                                                                 #
#                                                                 #
#      1. Realitzar consulta SQL                (No funcional)    #
#      2. Consultar els usuaris registrats      (READ)           #
#      3. Actualitzar el Nick d'un Usuari      (UPDATE CASCADE)  #
#      4. Donar de baixa un Usuari             (DELETE CASCADE)  #
#      5. Fer Administrador a un Usuari        (CREATE)          #
#      6. Consultar els administradors         (READ)            #
#      7. Sortir                                #                  #
#                                                                 #
#####

- Selecciona una opció: 
```

2. Consultar els usuaris registrats.

```
#####
#                                                                 #
#              ADMIN MENU  -  CRUD                               #
#                                                                 #
#####
#                                                                 #
#                                                                 #
#      1. Realitzar consulta SQL                (No funcional)    #
#      2. Consultar els usuaris registrats      (READ)           #
#      3. Actualitzar el Nick d'un Usuari      (UPDATE CASCADE)  #
#      4. Donar de baixa un Usuari             (DELETE CASCADE)  #
#      5. Fer Administrador a un Usuari        (CREATE)          #
#      6. Consultar els administradors         (READ)            #
#      7. Sortir                                #                  #
#                                                                 #
#####

- Selecciona una opció: 2

('Admin', 'Eric', 'Torrontera', 'Ruiz')
('Pedrita', 'Mila', 'La', 'Ella')
('Whiterose', 'Fatima', 'Ruiz', 'Arquellada')
```

2. Consultar els usuaris registrats.

```
elif opcio == 2:

    mycursor.execute("select NICK, Nom_Usuari, Cognom1, Cognom2 from Usuaris where NICK NOT LIKE 'Desconegut'")
    resultat = mycursor.fetchall()

    for element in resultat:
        print(element)
    print()
```

3. Actualitzar el Nick d'un Usuari.

```
Nick_current = input("          - Nick que vols actualitzar: ")

# Comprovem que existeix
mycursor.execute("select NICK from Usuaris WHERE NICK = %s", (Nick_current,))
res = mycursor.fetchone()

if res is None or res[0] == 'Desconegut':

    print("          - L'usuari no existeix, comprovi que el Nick és correcte! ")
    print()

else:

    # Si existeix continuem
    Nick_actualitzat = input("          - Nou Nickname: ")
    mycursor.execute("UPDATE Usuaris SET NICK = %s WHERE NICK = %s", (Nick_actualitzat, Nick_current,))
    mydb.commit()
    print("          - Nick actualitzat amb èxit!")
    print()
```

4. Donar de baixa un Usuari.

```
elif opcio == 4:

    Nick_current = input("          - Nick del Usuari que vols eliminar: ")

    # Comprovem que existeix
    mycursor.execute("select NICK from Usuaris WHERE NICK = %s", (Nick_current,))
    res = mycursor.fetchone()

    if res is None or res[0] == 'Desconegut' or res[0] == 'Admin':

        print("          - L'usuari no existeix, comprovi que el Nick és correcte! ")
        print()

    else:

        mycursor.execute("DELETE FROM Usuaris WHERE NICK = %s", (Nick_current,))
        mydb.commit()
        print("          - Nick eliminat amb èxit!")
        print()
```

5. Fer Administrador a un Usuari

```
elif opcio == 5:

    Nick_current = input("      - Nick del Usuari que vols afegir com admin: ")

    # Comprovem que existeix
    mycursor.execute("select NICK from Usuaris WHERE NICK = %s", (Nick_current,))
    res = mycursor.fetchone()

    if res is None or res[0] == 'Desconegut':

        print("      - L'usuari no existeix, comprovi que el Nick és correcte! ")
        print()
```

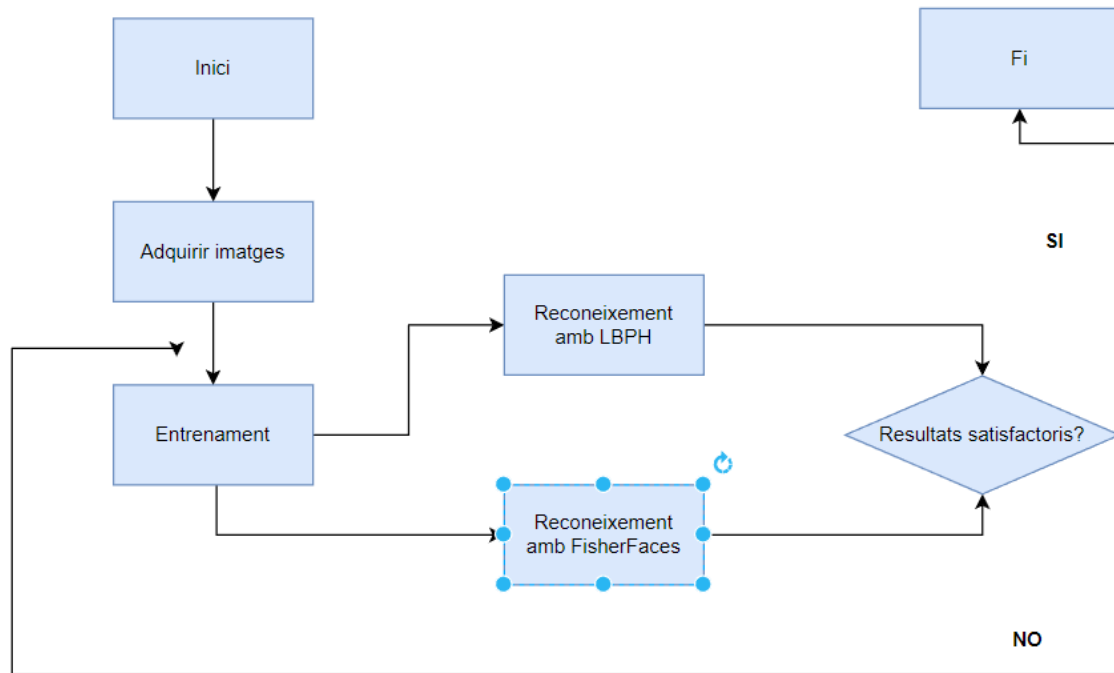
6.Consultar els administradors

```
elif opcio == 6:

    mycursor.execute("Select * from Administrador")
    resultat = mycursor.fetchall()

    for element in resultat:
        print(element)
    print()
```

METODOLOGÍA PER FER EL REONEIXEMENT FACIAL



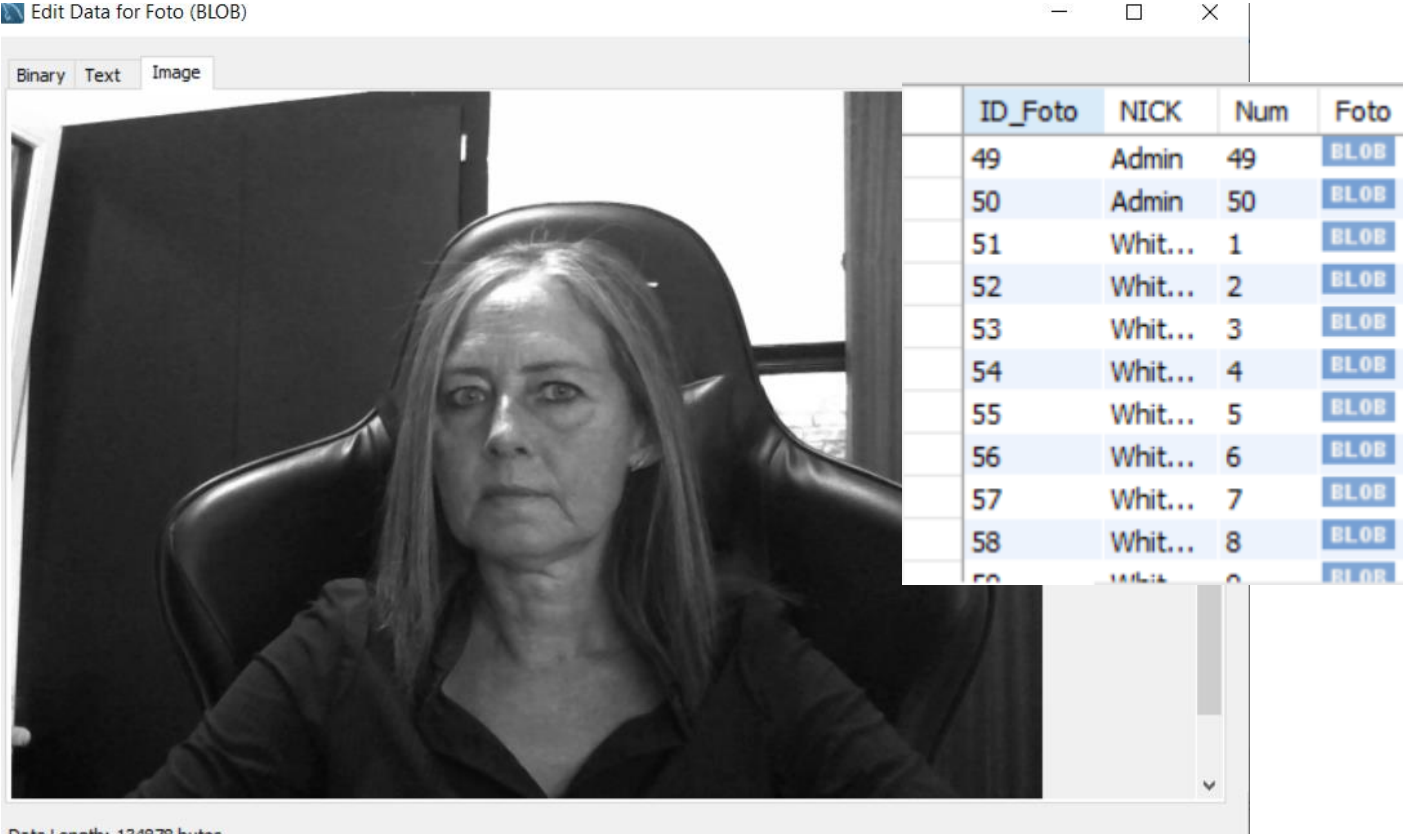
Per tal de fer la identificació del rostre hem utilitzat l'algoritme de detecció haarcascade_frontalcatface.xml existent en la nostra llibreria de OpenCV

Adquirir Imatges del Usuari

Primerament comentar que les funcions les he posat totes dintre de ReconeixementFacial.py

He definit **Crear_perfil()**. Aquesta funció el que fa principalment es recol·lectar les 50 imatges d'un usuari per tal de emplenar el Dataset. Durant el procés pesem la imatge a escala de grisos ja que l'algorisme que utilitzarem està basat en escala de grisos. Transformem les imatges a binari i les pugem a la nostra base de dades associat al Nick que hem registrat i amb el numero de imatge. Una vegada tenim les imatges pujades a MYSQL l'usuari podrà fer el login.

```
mycursor.execute("Insert into Dataset(NICK, Num, Foto) Values(%s, %s, %s)", (Usuari, Contador_imatges, data))  
mydb.commit()
```



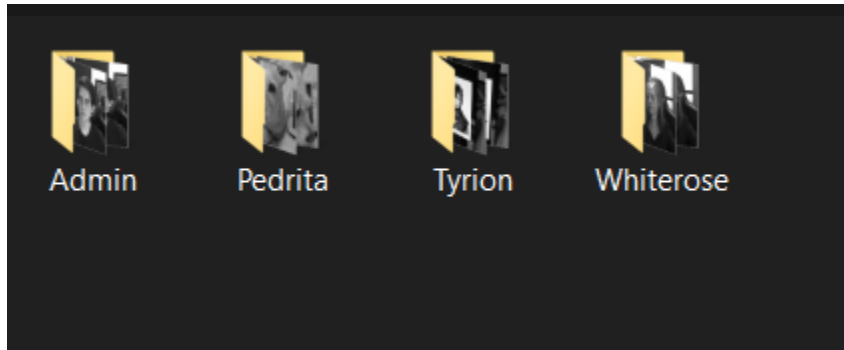
The screenshot shows a MySQL database interface window titled "Edit Data for Foto (BLOB)". The window has three tabs: "Binary", "Text", and "Image". The "Image" tab is selected, showing a grayscale photo of a woman with long hair sitting in a black leather chair. To the right of the image is a table with the following columns: ID_Foto, NICK, Num, and Foto. The table contains 10 rows of data, with the last row partially cut off.

ID_Foto	NICK	Num	Foto
49	Admin	49	BLOB
50	Admin	50	BLOB
51	Whit...	1	BLOB
52	Whit...	2	BLOB
53	Whit...	3	BLOB
54	Whit...	4	BLOB
55	Whit...	5	BLOB
56	Whit...	6	BLOB
57	Whit...	7	BLOB
58	Whit...	8	BLOB
59	Whit...	9	BLOB

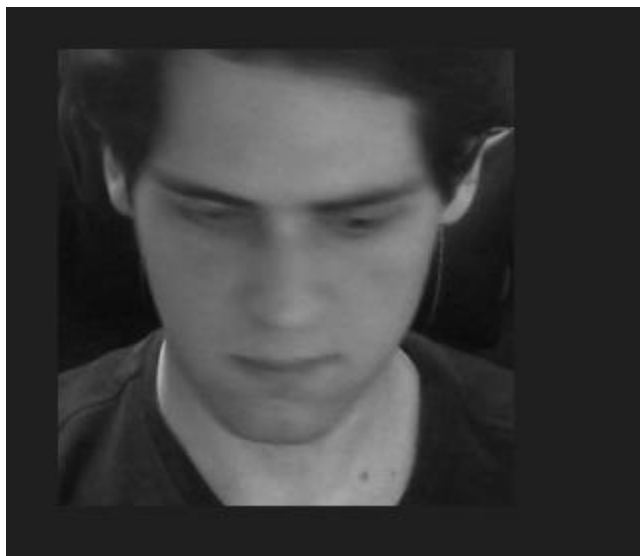
Data Length: 134878 bytes

Entrenament

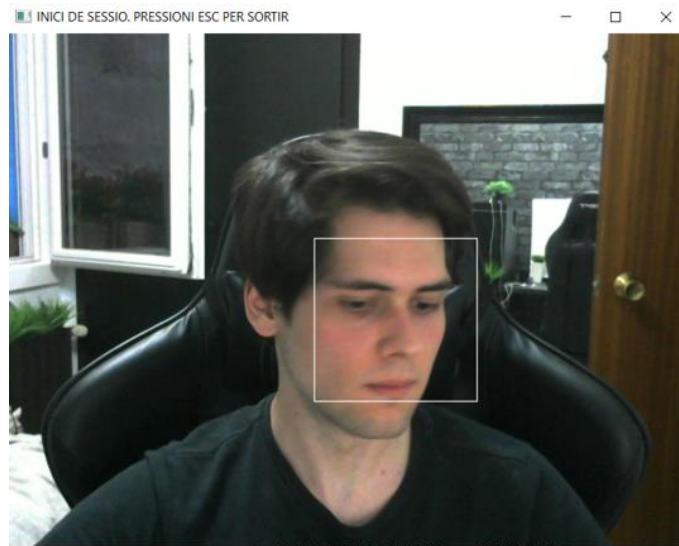
Tot seguit he definit **Load_MYSQL_Dataset()** que el que fa es baixar les imatges i posar-les en diferents carpetes per Nickname dintre del nostre directori de Splatoon, si no existeixen.



Tot seguit he definit **Crear_YML()** que per tal de fer la identificació del rostre utilitza l'algoritme de detecció haarcascade_frontalcatface.xml existent en la nostra llibreria de OpenCV. Aquesta funció utilitza la carpeta on hem guardat el nostre Dataset i extreu els nicks (etiquetes) les cares de les fotos i crea un arxiu .yaml que utilitzarem per fer el reconeixement facial.



Finalment la funció he creat la funció **reconeixement_facial(Nick)** que rep el Nick de l'usuari. Aquesta funció reconeix la cara de l'usuari i dibuixa un quadrat al seu voltant. Per reconèixer la cara utilitza haarcascade_frontalcatface.xml. La funció utilitza l'algorisme LBPH de cv2 que llegeix el nostre .yml que hem creat anteriorment. La funció detectarà una probabilitat de que la cara del usuari sigui una del nostre Dataset. Si la cara coincideix amb el Nick de l'usuari passarà el login.

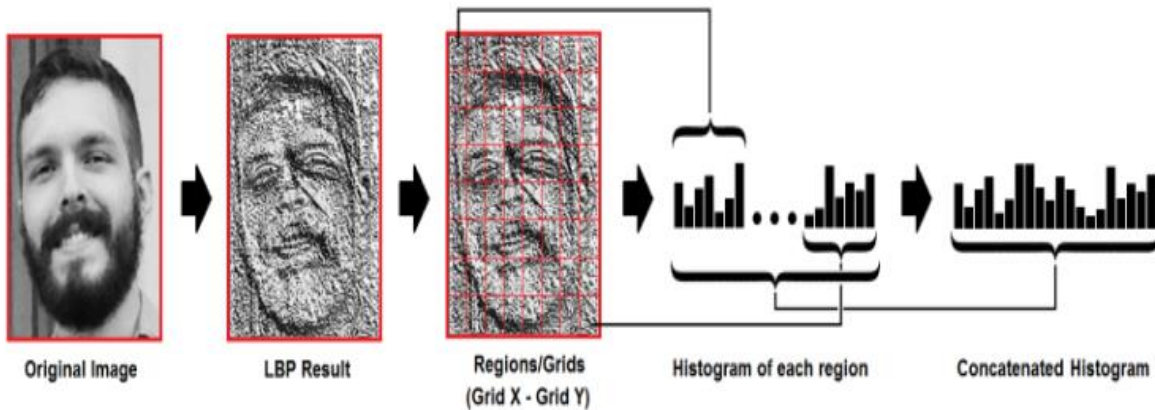


L'algoritme LBPH el que fa és:

1. Primer Converteix la imatge facial a escala de grisos.
2. Selecciona una finestra de 3x3 píxels. Serà una matriu de 3x3 que conté la intensitat de cada píxel.
3. Pren el valor central de la matriu.
4. Per a cada veí del valor central (lloc de resistència), establim un nou valor binari. S'estableix 1 per valors iguals o superiors al llindar i 0 per valors inferiors al llindar.
5. Ara, la matriu només conté valors binaris (ignorant el valor central). Hem de concatenar cada valor binari de cada posició de la matriu línia per línia en un nou valor binari (p. ex. 10001101).

Nota: alguns autors utilitzen altres mètodes per concatenar els valors binaris (per exemple, en el sentit de les agulles del rellotge), però el resultat final serà el mateix.

6. Després, convertim aquest valor binari en un valor decimal i el fixem en el valor central de la matriu, que en realitat és un píxel de la imatge original.
7. Al final d'aquest procediment (procediment LBP), tenim una nova imatge que representa millor les característiques de la imatge original.
8. Extreu l'histograma dels patrons LBP dividint la imatge en una graella.
9. Com que tenim una imatge en escala de grisos, cada histograma (de cada graella) només conté 256 posicions (0 a 255) que representen les ocurrències de cada intensitat de píxel.
10. Després, hem de concatenar cada histograma per crear un histograma nou i més gran. Suposant que tinguem xarxes 8x8, tindrem $8 \times 8 \times 256 = 16,384$ posicions en l'histograma final. L'histograma final representa les característiques de la imatge original.



Les imatges de les cares es comparen convertint ambdós en vectors LBPH i després calculant la distància entre dos histogrames, per exemple: la distància euclidiana,

L'algorisme FisherFaces el que fa és:

1. En primer lloc, recopilem un conjunt d'imatges. Les cares han de ser frontals, alineades i escapçades a la regió de la cara. Han de capturar suficients variacions d'il·luminació per a cada cara.
2. Converteix a escala de grisos i normalitza dividint cada píxel per 255. Llavors resta la cara mitjana (calculada pel conjunt de dades).
3. Calcular la matriu de covariància per al conjunt d'entrenament.
4. Calcular els valors propis i els vectors propis de la matriu de covariància - els vectors propis són la base ortonormal, és a dir, aquests són els components principals que defineixen l'espai de la cara.
5. Essencialment, les cares pròpies són dimensions al llarg de les quals s'han de calcular els components d'una cara donada. Els components resultants són les característiques extrems.
6. Utilitzar l'extractor de funcions apreses per extreure característiques de totes les imatges de cares.

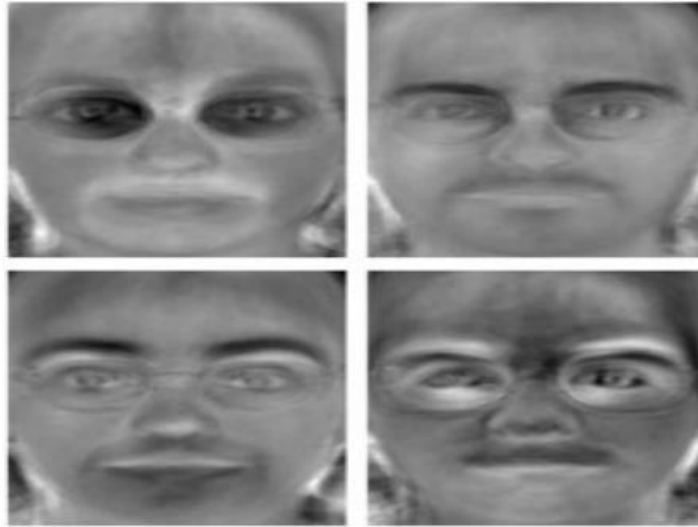


Figura 8: Conjunto de imagenes FisherFaces. (Martinez et al., 2018)

7. Finalment tiltza els vectors de característiques per realitzar la comparació.

Com el nostre treball es traca el reconeixement facial per tal d'entrar a una aplicació he escollit LBPH perquè és un mètode més fiable. FisherFaces no és prou precís per si mateix i necessita d'altres mètodes per a la millorar-lo.

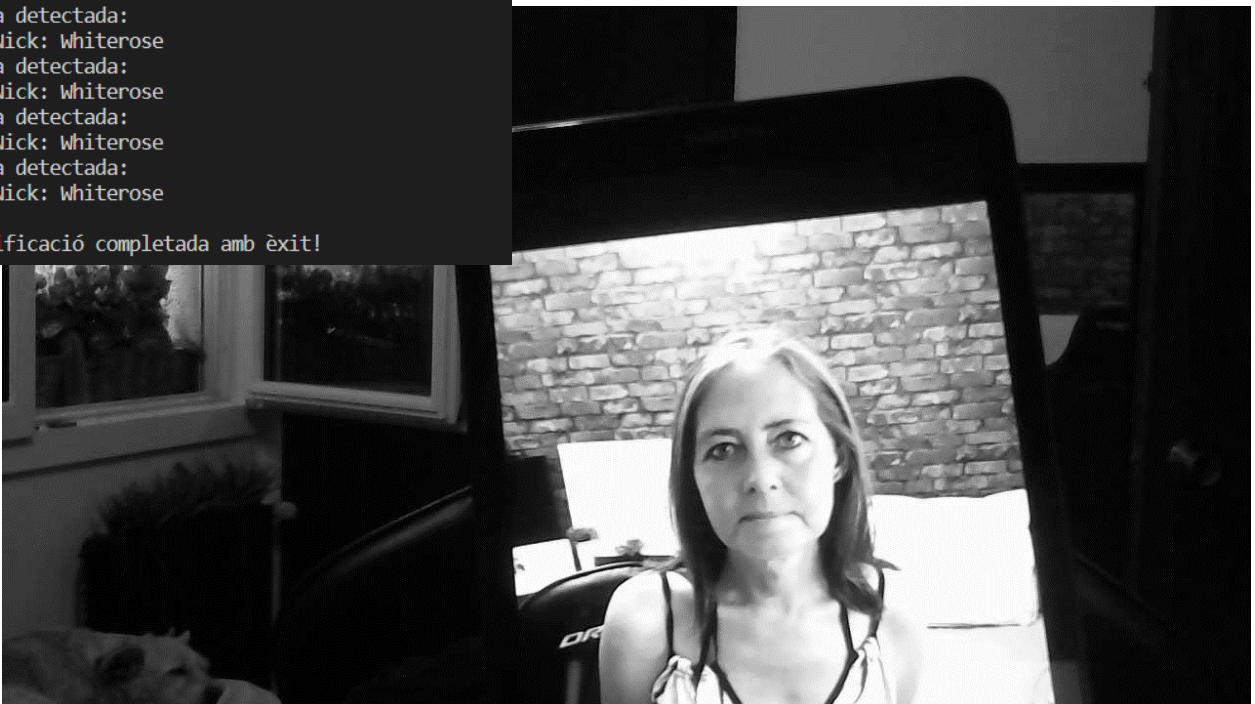
Resultats

```
print("    - Loading Data Credentials... Please Wait.")
ReconeixementFacial.Load_MYSQL_Dataset(mycursor)
print("    - Loading Done.")
print()
print("    - Creating YML... Please Wait.")
ReconeixementFacial.Crear_YML()
print("    - YML Created.")
print()
print("    - Starting facial recognition. Please wait.")
print()
Verificacio = ReconeixementFacial.reconeixement_facial(Nick)

if Verificacio and admin is not None:
    return True, Nick, True

if Verificacio:
    return True, Nick, False
```

```
- Starting facial recognition. Please wait.
+ Cara detectada:
  - Nick: Whiterose
+ Cara detectada:
  - Nick: Whiterose
+ Cara detectada:
  - Nick: Whiterose
+ Cara detectada:
  - Nick: Whiterose
+ Cara detectada:
  - Nick: Whiterose
+ Verificació completada amb èxit!
```



Altrament afegir que he deixat un fitxer .sql amb les inicials netes del projecte.

També he afegit ON UPDATE i DELETE CASCADE per tal de poder eliminar nicks i modificar-los en cascada. També que quan eliminem una partida eliminem tots els seus moviments de la nostra base de dades

```

> CREATE TABLE IF NOT EXISTS Usuaris (
  NICK                VARCHAR(75) NOT NULL UNIQUE,
  Nom_Usuari         VARCHAR(75) NOT NULL,
  Cognom1            VARCHAR(75),
  Cognom2            VARCHAR(75),
  Pw                 VARCHAR(39),
  Avatar             LONGBLOB,
  PRIMARY KEY (NICK));

> CREATE TABLE IF NOT EXISTS Dataset (
  ID_Foto             int AUTO_INCREMENT,
  NICK                VARCHAR(75),
  Num                 INT,
  Foto               LONGBLOB,
  PRIMARY KEY (ID_Foto),
  FOREIGN KEY (NICK) REFERENCES Usuaris (NICK) ON UPDATE CASCADE ON DELETE CASCADE
);

> CREATE TABLE IF NOT EXISTS Administrador (
  NICK                VARCHAR(75),
  Mes_Info           VARCHAR(70),
  PRIMARY KEY (NICK),
  FOREIGN KEY (NICK) REFERENCES Usuaris (NICK) ON UPDATE CASCADE ON DELETE CASCADE);

> CREATE TABLE IF NOT EXISTS Players (
  NICK                VARCHAR(75) NOT NULL UNIQUE,
  Info               VARCHAR(100),

```

Bibliografia i recursos.

Juárez Álvarez (Setembre del 2018) :RECONOCIMIENTO DE OBJETOS Y ROSTROS CON TÉCNICAS DE VISIÓN POR COMPUTADORA E INTELIGENCIA ARTIFICIAL[Arxiu PDF]. Proporcionat per la universitat d'Andorra. Data d'accés 20/06/22.

CodingEntrepreneurs (12 abr 2018) OpenCV Python TUTORIAL #4 for Face Recognition and Identification [Arxiu de Vídeo] <https://www.youtube.com/watch?v=PmZ29Vta7Vc&t=39s> Data d'accés 20/06/20.